فصل ۴: مدل‌های خطی

در این فصل، ما یک روش یادگیری ماشین برای رگرسیون خطی را در پیش خواهیم گرفت. توجه داشته باشید که این فصل، مشابه فصل کاهش ابعاد و PCA، شامل انتخاب بهترین ویژگی‌ها با استفاده از مدل‌های خطی است. حتی اگر تصمیم دارید از مدل‌های خطی برای پیش‌بینی‌ها استفاده کنید، می‌توانید قوی‌ترین ویژگی‌ها را انتخاب کنید. همچنین توجه داشته باشید که مدل‌های خطی درک الگوریتم‌های یادگیری ماشین را افزایش می‌دهند. به عنوان مثال، SVM با هسته RBF دارای مرزهای صاف می‌باشد، مانند یک خط به نظر می‌رسند. بنابراین، اگر در پس زمینه، شهود مدل خطی خود را به یاد بیاورید، توصیف SVMها اغلب آسان می‌شود.

عبور یک خط از داده‌ها

اکنون ما با برخی از مدل‌سازی‌های اساسی با رگرسیون خطی شروع خواهیم کرد. برای بسیاری از مردم این امر قابل فهم است که تغییر در یک متغیر ورودی، متغیر خروجی را تغییر می‌دهد. بسیاری از افراد در مدرسه، روزنامه، یا ارائه در محل کار و... این موارد را مشاهده کرده‌اند، و توضیح آن برای همکاران و سرمایه گذاران آسان خواهد بود.

مجموعه داده‌های بوستون برای آشنا شدن با رگرسیون بسیار مناسب است. این مجموعه داده دارای قیمت متوسط خانه در چندین منطقه‌ی بوستون است. همچنین عوامل دیگری را شامل می‌شود که ممکن است بر قیمت مسکن تأثیر بگذارد، به عنوان مثال، میزان جرم و جنایت.

شروع کنیم

- ابتدا مجموعه داده را وارد کنید .
- ابتدا شی LinearRegression را ایجاد کنید

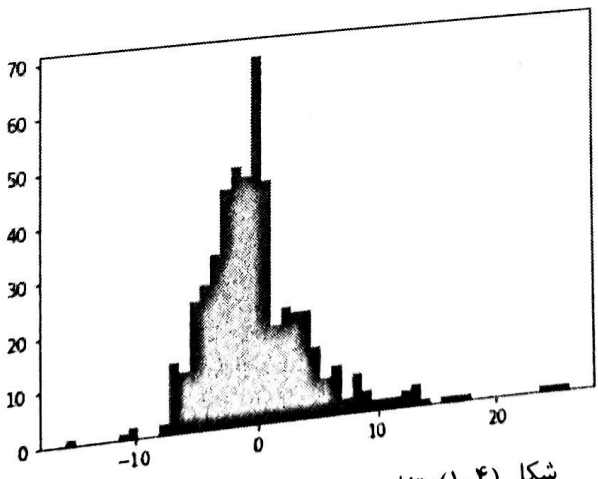
```
from sklearn import datasets
boston = datasets.load_boston()
from sklearn.linear_model import LinearRegression
lr = LinearRegression()
```

اکنون، به آسانی انتقال متغیرهای مستقل و وابسته را بر روی خطی برازش می‌کنیم و سپس داده‌های تست را پیش‌بینی می‌کنیم

```
lr.fit(boston.data, boston.target)
LinearRegression(copy_X=True, fit_intercept=True, n_jobs=1,
normalize=False)
predictions = lr.predict(boston.data)
```

پیش‌بینی‌های حاصل از رگرسیون خطی بدست آمد. حال، کلاس LinearRegression را کمی بیشتر کاوش می‌کنیم. به باقی‌مانده‌ها، تفاوت بین مجموعه هدف واقعی و مجموعه هدف پیش‌بینی شده نگاه کنید (شکل ۴-۱)

```
import numpy as np
import matplotlib.pyplot as plt
#within an Ipython notebook:
%matplotlib inline
plt.hist(boston.target - predictions , bins=50)
plt.show()
```



شکل (۴-۱): تفاوت مقدار هدف و پیش‌بینی شده

می‌توان ضرایب رگرسیون خطی را پیدا با دستور زیر مشاهده کنیم

```
lr.coef_
output: array([-1.08011358e-01, 4.64204584e-02, 2.05586264e-02,
2.68673382e+00,
   -1.77666112e+01, 3.80986521e+00, 6.92224640e-04, -1.47556685e+00,
   3.06049479e-01, -1.23345939e-02, -9.52747232e-01, 9.31168327e-03,
   -5.24758378e-01])
list(zip(boston.feature_names, lr.coef_))
output: [('CRIM', -0.10801135783679497),
('ZN', 0.046420458366879414),
('INDUS', 0.020558626367073864),
('CHAS', 2.6867338193449353),
('NOX', -17.7666112283001),
('RM', 3.809865206809222),
('AGE', 0.0006922246403407249),
('DIS', -1.4755668456002493),
('RAD', 0.30604947898516593),
('TAX', -0.012334593916574016),
('PTRATIO', -0.9527472317072905),
('B', 0.009311683273793638),
('LSTAT', -0.524758377855488)]
```

بنابراین می‌توان فهمید که کدام عوامل با نتیجه رابطه منفی دارند و همچنین عواملی که رابطه مثبت دارند. به عنوان مثال، همانطور که انتظار می‌رفت، افزایش در سرانه میزان جرم و جنایت توسط شهر رابطه منفی با قیمت یک خانه در بوستون دارد. سرانه جرم و جنایت اولین ضریب رگرسیون است. همچنین وقتی همه متغیرهای ورودی صفر هستند، می‌توانید مقدار intercept، مقدار پیش‌بینی شده هدف را مشاهده کنید

```
lr.intercept_
output: 36.45948838509005
```

نکته:

Intercept عرض از مبدا می‌باشد در فرمول y=vx+e اگر x برابر صفر باشد مقدار y برابر e می‌شود و همان intercept است.

ایده اصلی رگرسیون خطی یافتن مجموعه‌ای از ضرایب v است که $y = Xv$ را برآورده می‌کند، جایی که X ماتریس داده است. البته دور از انتظار است که برای مقادیر داده شده X، مجموعه‌ای از ضرایب را پیدا می‌کنیم که دقیقاً معادله را برآورده می‌کنند (یعنی به صورت تقریبی پیدا می‌شوند). البته یک متغیر خطا نیز در نظر گرفته می‌شود و معادله

به صورت y = Xv + c می‌شود، فرض می‌شود که توزیع c نرمال باشد و مستقل از X است. از نظر هندسی، این بدان معناست که خطا عمود بر X هستند.

LinearRegression می‌تواند ورودی‌ها را به طور خودکار نرمال کند

```
lr2 = LinearRegression(normalize=True)
lr2.fit(boston.data, boston.target)
LinearRegression(copy_X=True, fit_intercept=True, normalize=True)
predictions2 = lr2.predict(boston.data)
```

برازش (fit) خط از طریق داده‌ها

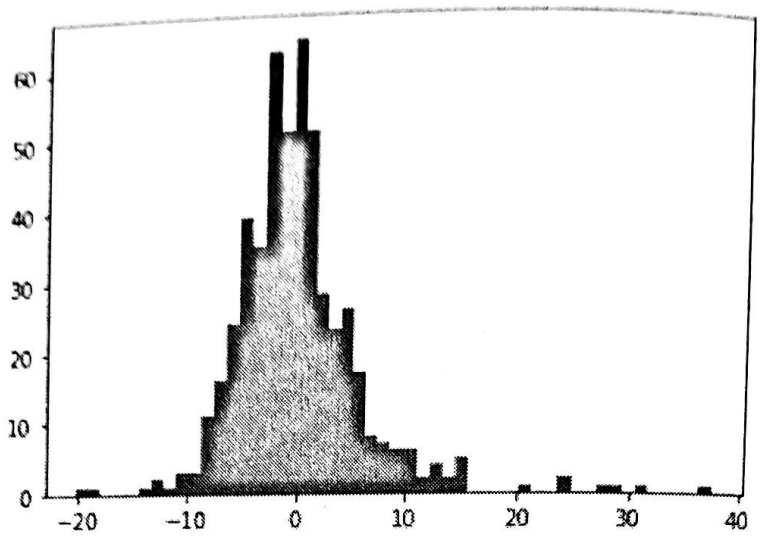
رگرسیون خطی با یادگیری ماشینی شامل آزمایش الگوریتم رگرسیون خطی بر روی داده‌های دیده نشده است. در اینجا، اعتبارسنجی 10-fold را انجام خواهیم داد

- مجموعه را به ۱۰ قسمت تقسیم کنید
- ۹ قسمت را آموزش دهید و قسمت باقیمانده را آزمایش کنید
- این کار را ۱۰ مرتبه تکرار کنید تا هر قسمت یکبار آزمایش شود
- همانند بخش قبل، مجموعه داده‌ای را که می‌خواهید رگرسیون خطی اعمال کنید، بارگذاری کنید. در این مورد، مجموعه داده مسکن بوستون است.
- متغیرهای مستقل و وابسته را به روش مناسب رگرسیون خطی منتقل کنید

```
from sklearn import datasets
boston = datasets.load_boston()
from sklearn.linear_model import LinearRegression
lr = LinearRegression()
lr.fit(boston.data, boston.target)
LinearRegression(copy_X=True, fit_intercept=True, n_jobs=1,
normalize=False)
```

حال 10-fold cross validation را اعمال می‌کنیم

```
from sklearn.model_selection import cross_val_predict
predictions_cv = cross_val_predict(lr, boston.data, boston.target,
cv=10)
import numpy as np
import matplotlib.pyplot as plt
#within Ipython
%matplotlib inline
plt.hist(boston.target - predictions_cv , bins=50)
plt.show()
```



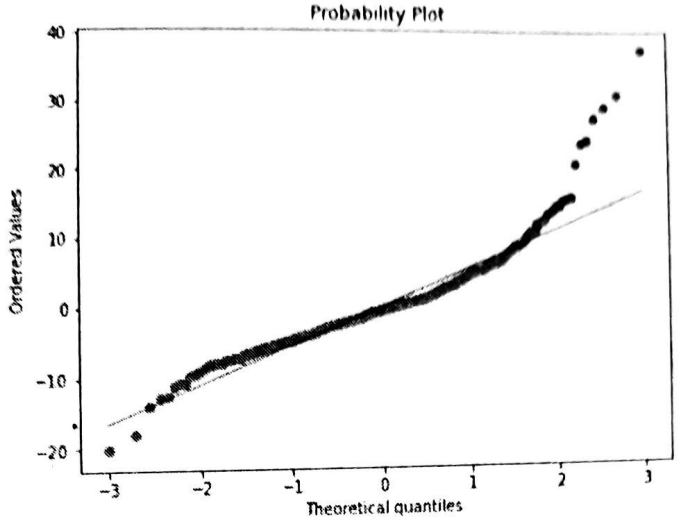
شکل- ۴-۲- اختلاف داده پیش‌بینی شده با هدف

با مشاهده باقیمانده‌ها (تفاوت بین داده‌های واقعی و پیش‌بینی‌ها) در مقایسه با باقیمانده‌های بخش قبلی رگرسیون خطی بدون اعتبارسنجی، دارای توزیع نرمال‌تری می‌باشند.

ارزیابی مدل رگرسیون خطی

در این دستورالعمل، بررسی خواهیم کرد که رگرسیون ما چقدر با داده‌های اساسی مطابقت دارد. ما از Scipy در اینجا استفاده خواهیم کرد زیرا یک تابع نمودار احتمال در داخل خود دارد که برای نمایش مناسب است.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import probplot
f = plt.figure(figsize=(7, 5))
ax = f.add_subplot(111)
tuple_out = probplot(boston.target - predictions_cv, plot=ax)
tuple_out[1]
output: (5.550500094062088, 0.03643470509167861 4, 0.9397036433372769)
```



شکل (۴-۳): یک مدل رگرسیون

این یک تکه از فرم (شیب، عرض از مبدا، r) است، که شیب و عرض از مبدا حاصل کمترین مربع‌ها، و r جذر ضرایب دترمینان است.

مقادیر اریب که قبلاً دیدیم، کمی واضح‌تر است. همچنین می‌توانیم برخی دیگر از معیارهای مناسب را بررسی کنیم؛ مانند میانگین مربع خطا (MSE) و میانگین انحراف مطلق (MAD) که دو معیار متداول هستند. بیایید هر یک را در تعریف و از آن‌ها استفاده کنیم.

```
def MSE(target, predictions):
 squared_deviation = np.power(target - predictions, 2)
 return np.mean(squared_deviation)

MSE(boston.target, predictions)
Output: 21.894831181729202
def MAD(target, predictions):
 absolute_deviation = np.abs(target - predictions)
 return np.mean(absolute_deviation)

MAD(boston.target, predictions)
Output: 3.2708628109003124
```

اکنون که فرمول‌ها را برای دریافت خطاها در NumPy مشاهده کرده‌اید، می‌توانید از ماژول sklearn.metrics نیز برای مشاهده سریع خطاها استفاده کنید.

```
from sklearn.metrics import mean_absolute_error, mean_squared_error
print ('MAE: ', mean_absolute_error(boston.target, predictions))
print ('MSE: ', mean_squared_error(boston.target, predictions))
```

output:
MAE: 3.2708628109003124
MSE: 21.894831181729202

فرمول MSE

$$\text{MSE} = \frac{1}{n}\sum_{1}^{n}(y_{truth} - y_{predict})^2$$

فرمول MAD

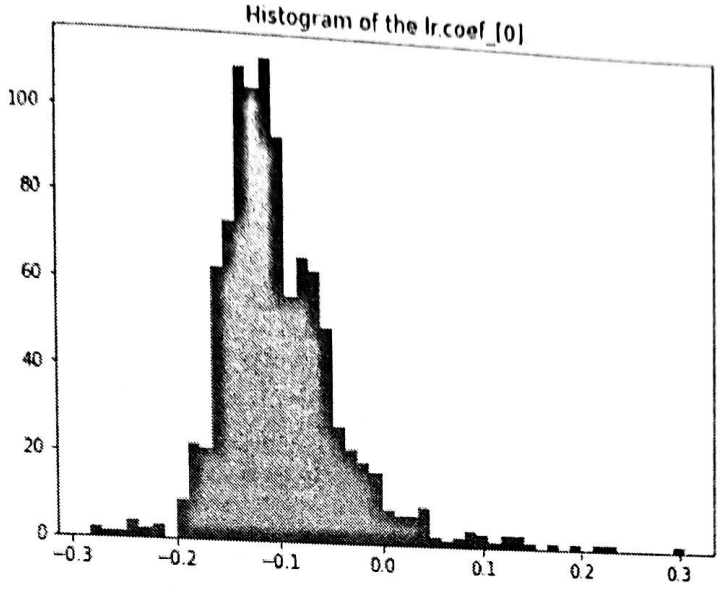
m میانگین است

$$\text{MAD} = \frac{1}{n}\sum_{1}^{n}(x_i - m)^2$$

استفاده از bootstarping

واقعیت این است که ضرایب متغیرهای تصادفی دارای توزیع می‌باشند. بیایید از bootstrapping برای بررسی توزیع ضریب میزان جرم استفاده کنیم. Bootstrapping یک روش بسیار ساده برای درک درستی از عدم اطمینان یک برآورد است.

```
n_bootstraps = 1000
len_boston = len(boston.target)
subsample_size = np.int(0.5*len_boston)
subsample = lambda: np.random.choice(np.arange(0,
len_boston),size=subsample_size)
coefs = np.ones(n_bootstraps) #pre-allocate the space for the coefs
for i in range(n_bootstraps):
  subsample_idx = subsample()
  subsample_X = boston.data[subsample_idx]
  subsample_y = boston.target[subsample_idx]
  lr.fit(subsample_X, subsample_y)
  coefs[i] = lr.coef_[0]
 import matplotlib.pyplot as plt
 f = plt.figure(figsize=(7, 5))
 ax = f.add_subplot(111)
 ax.hist(coefs, bins=50)
 ax.set_title("Histogram of the lr.coef_[0].")
```



شکل (۴-۴): coefهای رگرسیون logostic

حال فاصله اطمینان را مشاهده می‌کنیم

```
np.percentile(coefs, [2.5, 97.5])
output: array([-0.18623526, 0.04650185])
```

جالب است؛ در واقع دلیلی وجود ندارد که باور کنیم میزان جرم و جنایت ممکن است تاثیری بر قیمت خانه نداشته باشد. توجه کنید که صفر در فاصله اطمینان (CI) بین ۰/۱۸۶۲۳۵۲۶- و ۰/۰۴۶۵۰۱۸۵ ، به این معنی که ممکن است نقشی نداشته باشد. همچنین لازم به ذکر است که bootstraping به طور بالقوه می‌تواند منجر به برآورد بهتر ضرایب شود، زیرا میانگین محاسبه شده با همگرایی به میانگین واقعی سریع‌تر از یافتن ضریب با استفاده از تخمین زننده معمولی است.

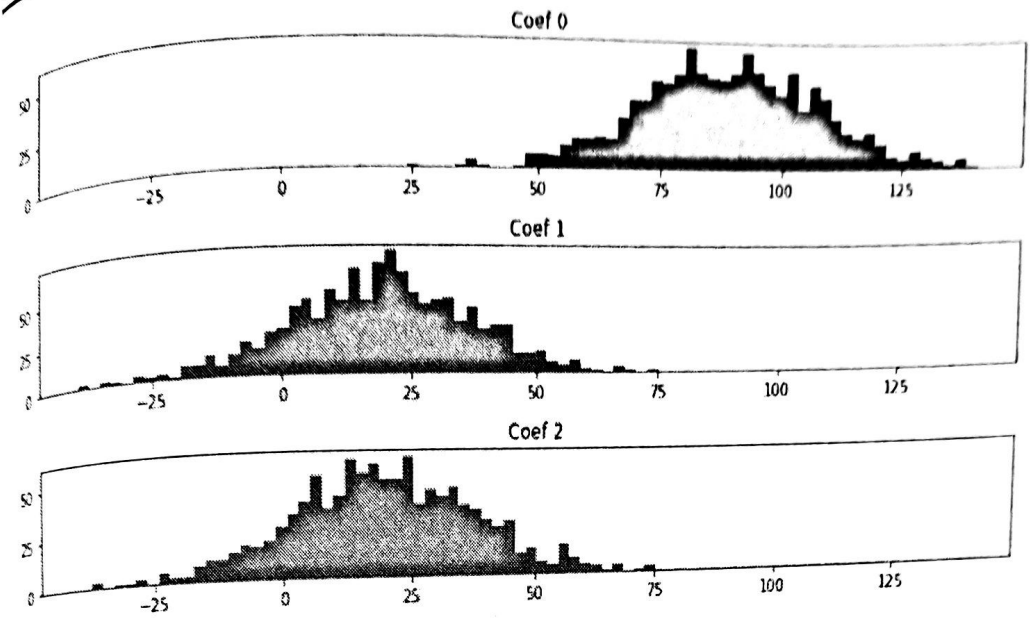
رگرسیون Ridge Regression

این روش با رگرسیون خطی وانیل متفاوت است و یک پارامتر منظم‌سازی برای ضرایب دارد. این زمانی مفید است که مجموعه داده دارای فاکتورهای هم‌خطی (در یک خط) باشد. رگرسیون ریج در واقع با همخطی آنقدر قدرتمند است که می‌توانید ویژگی‌های چند جمله‌ای را مدل کنید که همخطی و همبستگی زیادی دارند. در اینجا برای مقایسه رگرسیون ridge با رگرسیون خطی از طریق ضرایب، مجموعه داده‌ای را که دارای رتبه موثر پایین است استفاده می‌کنیم.

اول، از make_regression استفاده کنید تا یک مجموعه داده ساده با سه پیش‌بینی ایجاد کنید، و یک effective_rank را ۲ در نظر می‌گیریم. رتبه موثر به این معنی است که اگرچه از نظر فنی ماتریس دارای درجه کامل است، اما بسیاری از ستون‌ها دارای درجه همخطی بالایی هستند.

ابتدا از رگرسیون خطی معمولی استفاده می‌کنیم

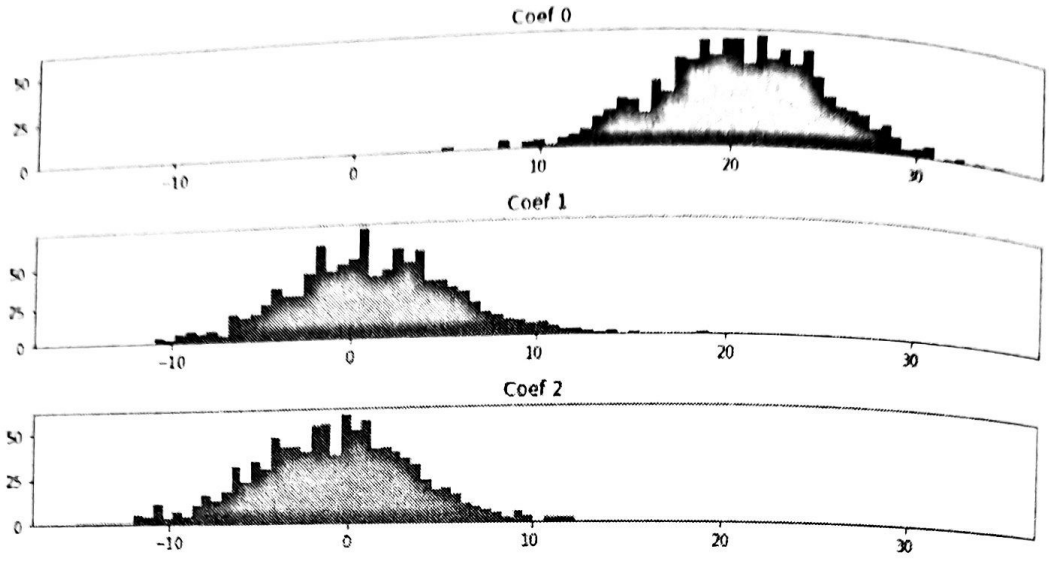
```
from sklearn.datasets import make_regression
reg_data, reg_target = make_regression(n_samples=2000,n_features=3,
effective_rank=2, noise=10)
import numpy as np
n_bootstraps = 1000
len_data = len(reg_data)
subsample_size = np.int(0.5*len_data)
subsample = lambda: np.random.choice(np.arange(0,
len_data),size=subsample_size)
coefs = np.ones((n_bootstraps, 3))
for i in range(n_bootstraps):
  subsample_idx = subsample()
  subsample_X = reg_data[subsample_idx]
  subsample_y = reg_target[subsample_idx]
  lr.fit(subsample_X, subsample_y)
  coefs[i][0] = lr.coef_[0]
  coefs[i][1] = lr.coef_[1]
  coefs[i][2] = lr.coef_[2]
import matplotlib.pyplot as plt
plt.figure(figsize=(10, 5))
ax1 = plt.subplot(311, title ='Coef 0')
ax1.hist(coefs[:,0], bins=50)
ax2 = plt.subplot(312,sharex=ax1, title ='Coef 1')
ax2.hist(coefs[:,1], bins=50)
ax3 = plt.subplot(313,sharex=ax1, title ='Coef 2')
ax3.hist(coefs[:,2] , bins=50)
plt.tight_layout()
```



شکل (۴-۵): مقایسه انواع coef

حال رگرسیون ridge را بر روی همان داده‌ها انجام می‌دهیم

```
from sklearn.linear_model import Ridge
r = Ridge()
n_bootstraps = 1000
len_data = len(reg_data)
subsample_size = np.int(0.5*len_data)
subsample = lambda: np.random.choice(np.arange(0,
len_data),size=subsample_size)
coefs_r = np.ones((n_bootstraps, 3))
for i in range(n_bootstraps):
 subsample_idx = subsample()
 subsample_X = reg_data[subsample_idx]
 subsample_y = reg_target[subsample_idx]
 r.fit(subsample_X, subsample_y)
 coefs_r[i][0] = r.coef_[0]
 coefs_r[i][1] = r.coef_[1]
 coefs_r[i][2] = r.coef_[2]
plt.figure(figsize=(10, 5))
ax1 = plt.subplot(311, title ='Coef 0')
ax1.hist(coefs_r[:,0], bins=50)
ax2 = plt.subplot(312,sharex=ax1, title ='Coef 1')
ax2.hist(coefs_r[:,1], bins=50)
ax3 = plt.subplot(313,sharex=ax1, title ='Coef 2')
ax3.hist(coefs_r[:,2], bins=50)
plt.tight_layout()
```



شکل (۴-۶): مقایسه coefها

ضرایب رگرسیون Ridge بسیار نزدیک به صفر هستند

حال به ضرایب رگرسیون معمولی نگاه بیندازید

```
np.var(coefs, axis=0)
output: array([331.37644628, 301.78782708, 318.01481892])
```

حال به ضرایب رگرسیون ridge نگاه بیندازید

```
np.var(coefs_r, axis=0)
array([19.31369862, 19.88765348, 19.46988906])
```

بنابراین، به طور متوسط ضرایب رگرسیون خطی بسیار بیشتر از ضرایب رگرسیون ridge هستند؛ و این اختلاف در ضرایب، بایاس است. واریانس به طور چشمگیری کاهش یافته است. این توازن واریانس- بایاس است که اغلب در یادگیری ماشین مورد بحث قرار می‌گیرد.

بهینه‌سازی پارامتر رگرسیون Ridge

انتخاب پارامتر آلفا هنگام استفاده از روش Ridge برای پیش‌بینی یا یادگیری روابط در سیستم باید مورد توجه قرار گیرد. به عنوان مثال، استفاده از رگرسیون حداقل مربعات معمولی (OLS) ممکن است رابطه‌ای بین دو متغیر را نشان دهد. با این حال، هنگامی که توسط یک آلفا تنظیم می‌شود، رابطه معنی دار نمی‌باشد.

از طریق cross-validation، پارامتر آلفای رگرسیون ridge تنظیم خواهیم کرد. در رگرسیون ridge، پارامتر گاما معمولاً هنگام فراخوانی RidgeRegression به نام آلفا نمایش داده می‌شود. بنابراین، سوالی که پیش می‌آید این است که بهترین آلفا کدام است؟ یک مجموعه داده رگرسیون ایجاد و سپس شروع می‌کنیم.

RidgeCV که مخفف ridge cross-validation است که اعتبارسنجی متقابل مانند leave-one-out cross-validation (LOOCV) را انجام می‌دهد.

شروع کنیم

- این مدل برای همه نمونه‌ها به جز یک مدل آموزش داده و سپس خطای پیش‌بینی این مورد آزمایشی را ارزیابی می‌کنیم
- بعد از اینکه رگرسیون برازش شد، آلفا محاسبه شده بهترین انتخاب خواهد بود

```
from sklearn.datasets import make_regression
reg_data, reg_target = make_regression(n_samples=100, n_features=2,
effective_rank=1, noise=10)
from sklearn.linear_model import RidgeCV
rcv = RidgeCV(alphas=np.array([.1, .2, .3, .4]))
rcv.fit(reg_data, reg_target)
rcv.alpha_
output: 0.1
```

در هر مرحله از فرآیند اعتبارسنجی متقابل، مدل خطایی را در برابر نمونه آزمون به دست می‌آورد. به طور پیش فرض، در اصل یک خطای مربع است. RidgeCV می‌تواند مقادیر اعتبارسنجی را ذخیره کند. این به ما اجازه می‌دهد آنچه را انجام می‌دهد تصویر کنیم.

همانطور که مشاهده می‌کنید، ما یک دسته از نقاط (در کل ۵۰) را بین ۰.۰۱ تا ۱ آزمایش می‌کنیم.

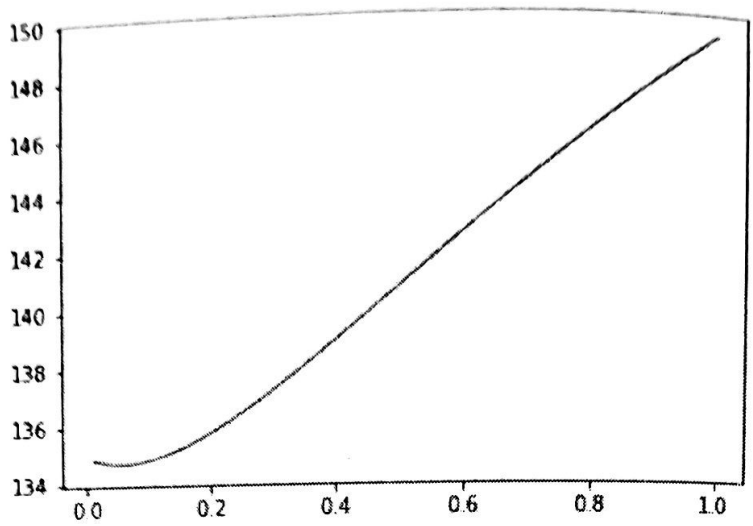
```
alphas_to_test = np.linspace(0.01, 1)
rcv3 = RidgeCV(alphas=alphas_to_test, store_cv_values=True)
rcv3.fit(reg_data, reg_target)
rcv3.cv_values_.shape
output: (100, 50)
```

بنابراین، ما در رگرسیون اولیه ۱۰۰ مقدار داشتیم و ۵۰ مقدار مختلف آلفا را آزمایش کردیم. اکنون به خطاهای تمام ۵۰ مقدار دسترسی داریم که می‌توانیم کوچک‌ترین خطای میانگین را پیدا کرده و آن را به عنوان آلفا انتخاب کنیم.

```
smallest_idx = rcv3.cv_values_.mean(axis=0).argmin()
alphas_to_test[smallest_idx]
output: 0.050408163265306122
```

این مقدار با بهترین مقداری که توسط نمونه rcv3 کلاس RidgeC پیدا شده مطابقت دارد

```
rcv3.alpha_
output: 0.050408163265306122
plt.plot(alphas_to_test, rcv3.cv_values_.mean(axis=0))
```



شکل (۴-۷): مقدار آلفا و خطای میانگین

اگر بخواهیم از تابع امتیازدهی خودمان استفاده کنیم، می‌توانیم این کار را نیز انجام دهیم. در این مثال از MAD برای امتیاز دهی استفاده می‌کنیم. ابتدا باید تابع خطای (loss function) خود را تعریف کنیم. بعد از اینکه تابع خطا را تعریف کردیم، می‌توانیم از make_scorer در sklearn استفاده کنیم که به استانداردسازی عمل ما کمک می‌کند.

```
from sklearn.metrics import mean_absolute_error
from sklearn.metrics import make_scorer
MAD_scorer = make_scorer(mean_absolute_error, greater_is_better=False)
rcv4 = RidgeCV(alphas=alphas_to_test, store_cv_values=True,
scoring=MAD_scorer)
rcv4.fit(reg_data, reg_target)
smallest_idx = rcv4.cv_values_.mean(axis=0).argmin()
rcv4.cv_values_.mean(axis=0)[smallest_idx]
outout: 0.017103872819757095
alphas_to_test[smallest_idx]
output: 1.0
```

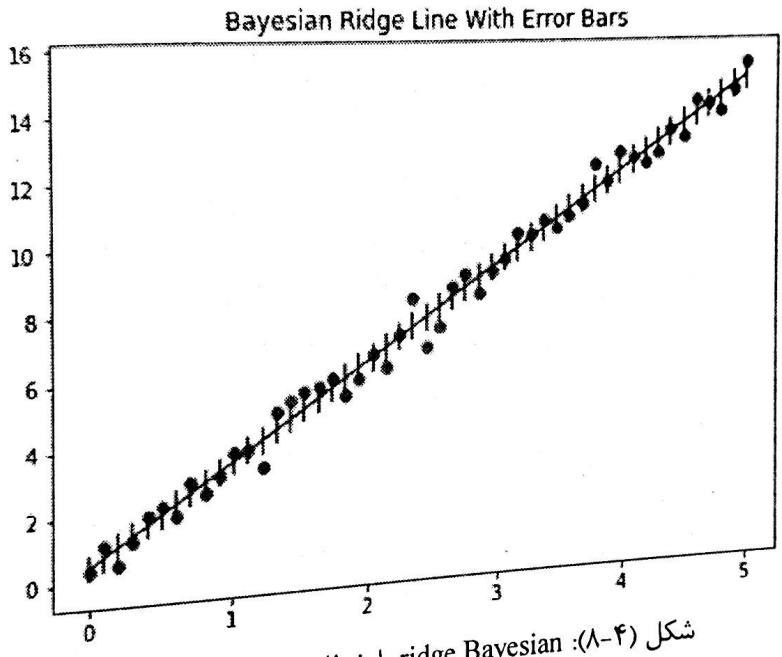
رگرسیون ridge بیز

علاوه بر این، scikit-learn حاوی رگرسیون ridge بیزی است که امکان تخمین فواصل اطمینان را می‌دهد.

یک خط با شیب ۳ و بدون عرض از مبدا ایجاد کنید(y=3X)

```
X = np.linspace(0, 5)
y_truth = 3 * X
```

```
y_noise = np.random.normal(0, 0.5, len(y_truth)) #normally distributed noise
with mean 0 and spread 0.1
y_noisy = (y_truth + y_noise)
from sklearn.linear_model import BayesianRidge
br_inst = BayesianRidge().fit(X.reshape(-1, 1), y_noisy)
y_pred, y_err = br_inst.predict(X.reshape(-1, 1), return_std=True)
plt.figure(figsize=(7, 5))
plt.scatter(X, y_noisy)
plt.title("Bayesian Ridge Line With Error Bars")
plt.errorbar(X, y_pred, y_err, color='green')
```



شکل (۴-۸): ridge Bayesian با خطاهای مشخص شده

استفاده از پراکندگی برای منظم‌سازی مدل‌ها

متد least absolute shrinkage and selection operator (LASSO) بسیار شبیه به رگرسیون ridge و least angle regression (LARS) است. این همانند رگرسیون ridge است به این معنا که ما رگرسیون خود را با مقادیری جریمه می‌کنیم، همچنین مانند LARS می‌تواند به عنوان یک پارامتر مورد استفاده قرار گیرد. ویژگی‌هایی که نیاز نیست به مجموعه داده وابسته باشد توسط LASSO و LARS نادیده گرفته می‌شوند. (از طرف دیگر، رگرسیون ridge تمام ویژگی‌ها را حفظ می‌کند، که به شما امکان می‌دهد توابع چند جمله‌ای یا توابع پیچیده را با ویژگی‌های همبسته مدلسازی کنید).

استفاده ازLASSO می‌تواند پیامدهای محاسباتی داشته باشد. همانطور که در این دستورالعمل خواهیم دید، ما از یک تابع خطا استفاده خواهیم کرد که نیاز به راه حل‌های ویژه دارد.

از make_regression استفاده می‌کنیم و یک مجموعه داده با همان پارامترها ایجاد می‌کنیم

```
import numpy as np
from sklearn.datasets import make_regression
reg_data, reg_target = make_regression(n_samples=200,
n_features=500, n_informative=5, noise=5)
from sklearn.linear_model import Lasso
lasso = Lasso()
```

LASS شامل پارامترهای بسیاری است، اما جالب‌ترین پارامتر آلفا است که جریمه Lasso را مقیاس‌بندی می‌کند. اگر صفر باشد، LASSO معادل رگرسیون خطی است.

```
lasso.fit(reg_data, reg_target)
np.sum(lasso.coef_ != 0)
output: 7
lasso_0 = Lasso(0)
lasso_0.fit(reg_data, reg_target)
np.sum(lasso_0.coef_ != 0)
output: 500
```

Lasso cross-validation

ما می‌توانیم خود آلفا را مشخص کنیم یا از cross-validation برای یافتن بهترین مقدار با توجه به داده‌های موجود استفاده کنیم.

```
from sklearn.linear_model import LassoCV
lassocv = LassoCV()
lassocv.fit(reg_data, reg_target)
lassocv.alpha_
output: 0.7365860481835312
lassocv.coef_[:5]
output: array([ 0., -0., 0., -0., -0.])
np.sum(lassocv.coef_ != 0)
output:21
```

LASSO برای انتخاب ویژگی

LASSO اغلب در انتخاب ویژگی برای روش‌های دیگر استفاده می‌شود. به عنوان مثال، ممکن است LASSOregression را برای بدست آوردن تعداد مناسب ویژگی‌ها اجرا کرده و

سپس این ویژگی‌ها در الگوریتم دیگری استفاده کنید. برای بدست آوردن ویژگی‌های مورد نظر خود، یک آرایه mask بر اساس ستون‌هایی که صفر نیستند ایجاد کنید و سپس ستون‌های غیر صفر را فیلتر کنید تا ویژگی‌های مورد نظر ما حفظ شود.

```
mask = lassocv.coef_ != 0
new_reg_data = reg_data[:, mask]
new_reg_data.shape
```

اتخاذ رویکرد اساسی‌تری برای قاعده مندسازی با LARS

LARS یک روش رگرسیون است که به خوبی برای مسائل با ابعاد بالا، یعنی p >> n مناسب است، جایی که p ستون‌ها یا ویژگی‌ها را نشان می‌دهد وn تعداد نمونه‌ها را نشان می‌دهد.

- داده‌هایی که ما استفاده می‌کنیم دارای ۲۰۰ نقطه داده و ۵۰۰ ویژگی خواهد بود.
- از آنجا که ما از ۱۰ ویژگی مناسب استفاده کردیم، پس ۱۰ ضریب غیر صفر در LARS می‌خواهیم.
- سپس می‌توانیم بررسی کنیم که LARS تعداد صحیح ضرایب غیر صفر را برمی‌گرداند

```
from sklearn.datasets import make_regression
reg_data, reg_target = make_regression(n_samples=200,
n_features=500, n_informative=10, noise=2)

from sklearn.linear_model import Lars
lars = Lars(n_nonzero_coefs=10)
lars.fit(reg_data, reg_target)
np.sum(lars.coef_ != 0)
output: 10
```

سوال این است که چرا استفاده از تعداد کمتری از ویژگی‌ها مفیدتر است؟ برای نشان دادن این مسئله، بیایید نیمی از داده‌ها را نگه داشته و دو مدل LARS را آموزش دهیم، یکی با ۱۲ ضریب غیر صفر و دیگری بدون مقدار از پیش تعیین شده.

```
train_n = 100
lars_12 = Lars(n_nonzero_coefs=12)
lars_12.fit(reg_data[:train_n], reg_target[:train_n])
lars_500 = Lars() # it's 500 by default
lars_500.fit(reg_data[:train_n], reg_target[:train_n]);
np.mean(np.power(reg_target[train_n:] -lars_12.predict(reg_data[train_n:]), 2))
# Error function
output: 80.30998224234564
```

```
np.mean(np.power(reg_target[train_n:] -lars_500.predict(reg_data[train_n:]),
2))# Error function
output: 2.0638094716161772e+32
```

همان طور که مشاهده می‌کنید خطای ۵۰۰ ویژگی بیشتر از خطای ۱۲ ویژگی می‌باشد. در اینجا مشکل مجموعه داده‌های با ابعاد بالا وجود دارد. با توجه به تعداد زیادی ویژگی‌ها، بیش برازش به یک مشکل بزرگ تبدیل می‌شود. روش LARS با انتخاب تکراری ویژگی‌هایی که با باقیمانده در ارتباط هستند کار می‌کند. از لحاظ هندسی، همبستگی در واقع کمترین زاویه بین ویژگی و باقیمانده است. پس از انتخاب اولین ویژگ، LARS به حرکت در کمترین زاویه ادامه خواهد داد تا زمانی که یک ویژگی دیگر با باقیمانده‌ها یکسان همبستگی داشته باشد. سپس، LARS در جهت ترکیبی هر دو ویژگی شروع به حرکت می‌کند.

تقریباً به همان روشی که از cross-validation برای تنظیم ridge رگرسیون استفاده کردیم، می‌توانیم همین کار را با LARS انجام دهیم. استفاده از اعتبارسنجی متقابل به ما کمک می‌کند تا بهترین تعداد ضرایب غیر صفر را تعیین کنیم.

```
from sklearn.linear_model import LarsCV
lcv = LarsCV()
lcv.fit(reg_data, reg_target)
np.sum(lcv.coef_ != 0)
output: 9
```